Manual de despliegue del clasificador de tumores

Clasificador de tumores · Fase II · Modelo 1.2.0
Actualización del 1 de noviembre de 2026

Este manual permite reproducir la ejecución del frontend Nginx y la API FastAPI mediante Docker Compose y trasladar el mismo proyecto a una máquina virtual en nube. La ejecución local quedó comprobada con contenedores saludables, nueve pruebas aprobadas y una predicción desde el navegador. El despliegue en nube se describe como procedimiento; no se acredita una publicación real en nube.

Arquitectura y requisitos

El navegador solicita /api/predict al frontend. Nginx elimina el prefijo /api/ y reenvía la solicitud a backend:8000/predict. FastAPI valida 30 características y ejecuta el pipeline de imputación, escalado y regresión logística versionado como 1.2.0.

Elemento	Configuración
Repositorio	https://github.com/HamDan314/FAseII.1
Servicios	backend y frontend
Puertos predeterminados	API 8000 y web 8080
Requisitos locales	Docker Desktop con contenedores Linux y Compose v2 con --wait
Archivos	backend/Dockerfile, frontend/Dockerfile y docker-compose.yml

Ejecución local en Windows

Descargar el ZIP de main y descomprimirlo por completo. Abrir Docker Desktop y ejecutar INICIAR_DOCKER.bat. El archivo construye las imágenes, espera el estado saludable, ejecuta pytest y comprueba la integración. No es necesario volver a entrenar el modelo.

http://localhost:8080
http://localhost:8000/docs

Si existe un contenedor de una ejecución anterior con el mismo nombre, eliminar únicamente los contenedores anteriores de este proyecto y volver a ejecutar el archivo BAT. El comando detiene y elimina esos dos contenedores.

```
docker rm -f cancer-api cancer-frontend
```

Despliegue en una máquina virtual en nube

Utilizar una VM Linux con Docker Engine, el complemento Docker Compose v2 y Git instalados. Como punto de partida para esta demostración, asignar 2 vCPU y 4 GB de RAM; ajustar tras observar el consumo. Disponer de acceso SSH, permisos para Docker y salida de red para descargar imágenes y dependencias.

1. Crear la VM en el proveedor elegido y conectarse mediante SSH. Restringir el acceso SSH a la IP del operador. Instalar Docker y Git según las instrucciones oficiales de la distribución y verificar docker version y docker compose version.

2. Descargar una revisión reproducible del proyecto. El siguiente commit contiene la aplicación comprobada; los documentos de esta actualización se publican después de esa revisión.

```
git clone https://github.com/HamDan314/FAseII.1.git
cd FAseII.1
git checkout d883fb7e5eaa43768caa57a5eae352c31743bd06
```

3. Para restringir inicialmente el servicio a la propia VM, cambiar los mapeos ports de docker-compose.yml por los siguientes valores. No modificar los puertos internos de los contenedores.

```
# backend > ports
- "127.0.0.1:8000:8000"
# frontend > ports
- "127.0.0.1:8080:80"
```

4. Construir, iniciar y comprobar los servicios desde la carpeta del proyecto. Ante un error, consultar los logs antes de continuar.

```
docker compose config --quiet
docker compose up -d --build --wait --wait-timeout 180
docker compose ps
docker compose exec -T backend python -m pytest -q
docker compose exec -T backend python tests/comprobar_integracion.py
curl --fail http://127.0.0.1:8080/api/health
```

5. Para una demostración privada, crear un túnel desde la computadora del operador. Sustituir USUARIO e IP_VM por los valores reales; si el puerto local 8080 está ocupado, elegir otro en el lado izquierdo.

```
ssh -L 8080:127.0.0.1:8080 USUARIO@IP_VM
```

Con el túnel abierto, acceder a http://localhost:8080. Para publicación pública, configurar un dominio y un proxy de entrada con HTTPS hacia 127.0.0.1:8080. Mantener 8000 y 8080 cerrados en el firewall externo; abrir 443 y, si el mecanismo de certificados lo requiere, 80. Validar la URL HTTPS antes de compartirla.

Operación y evidencia de funcionamiento

Comprobaciones y recuperación

docker compose logs --tail=80
docker compose restart
docker compose down

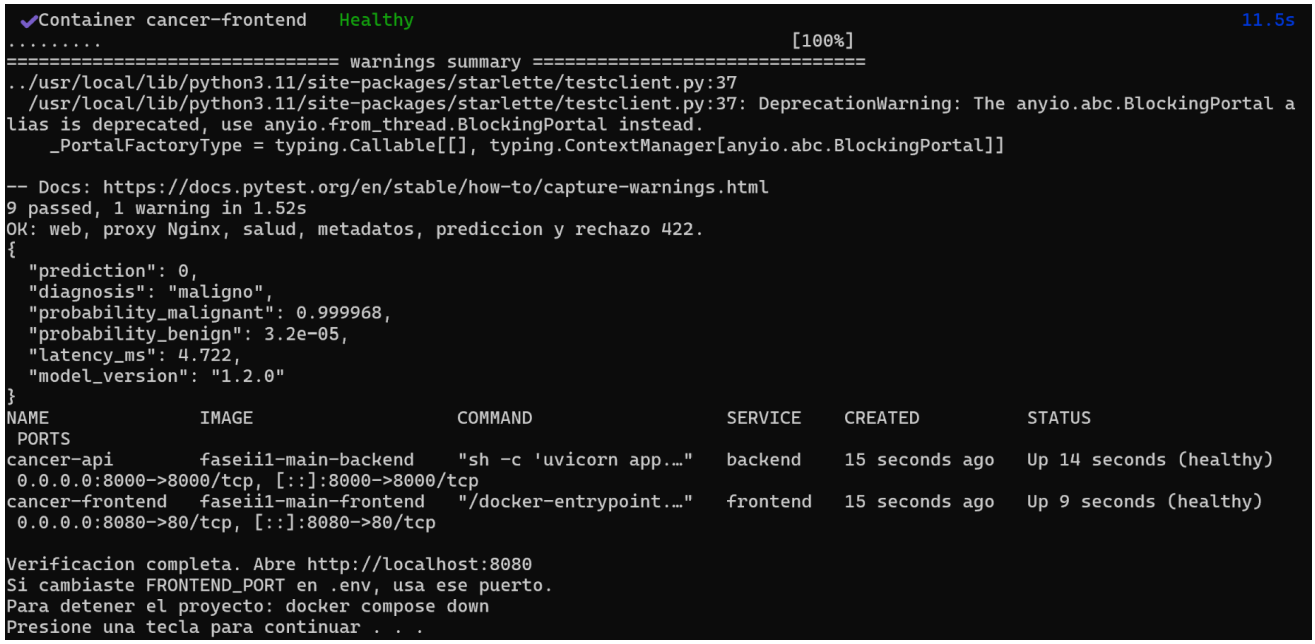
Para actualizar, conservar la revisión anterior, obtener la revisión deseada, reconstruir con docker compose up -d --build --wait y repetir las comprobaciones. Para volver atrás, seleccionar el commit anterior y reconstruir. Cambiar FRONTEND_PORT o BACKEND_PORT en .env cuando haya conflicto de puertos en ejecución local.

La API no implementa cuentas, contraseñas, sesiones ni tokens. CORS no sustituye autenticación. El servicio público necesita control de acceso si se exige restringir usuarios. No registrar datos clínicos reales; este sistema es académico. La carga de joblib debe limitarse a artefactos confiables del proyecto.

La configuración Nginx incluida resuelve el nombre backend de Docker Compose. Publicar frontend y backend como servicios independientes exige modificar el proxy o retirar su dependencia de ese nombre; cambiar solamente API_URL en JavaScript no resuelve el arranque de Nginx.

Resultado local observado

Captura aportada de la ejecución en Windows con Docker Desktop: nueve pruebas aprobadas, integración correcta y ambos contenedores saludables. Se observó una advertencia de deprecación de AnyIO que no causó fallos. Esta evidencia no acredita despliegue en nube.



Fuentes del proyecto: README.md, INICIAR_DOCKER.bat, docker-compose.yml, backend/app/main.py, frontend/nginx.conf y docs/AUTENTICACION.md. Código revisado: d883fb7e5eaa43768caa57a5eae352c31743bd06.